

Is 분석 보고서

이름: 신희수

0. ls 명령어에 사용된 시스템 콜을 알아보고 보고서 작성

a. ls 명령어에서 호출한 시스템콜 목록

```
m2ntch0co@m2ntch0co-VirtualB8xi:~$ strace ls
execve("/usr/bin/ls", ["ls"], 0x7ffffce05650 /* 44 vars */) = 0
brk(NULL)                               = 0x63730a27b000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffe69468970) = -1 EINVAL (부적절한 인수)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x795767613000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (그런 파일이나 디렉터리가 없습니다)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=57875, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 57875, PROT_READ, MAP_PRIVATE, 3, 0) = 0x795767604000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0-\0\1\0\0\0\0\0\0\0\0\0\0....", 832) = 832
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=166280, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 177672, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7957675d8000
mprotect(0x7957675de000, 139264, PROT_NONE) = 0
mmap(0x7957675de000, 106496, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6000) = 0x7957675de000
mmap(0x7957675f8000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x20000) = 0x7957675f8000
mmap(0x795767600000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x27000) = 0x795767600000
mmap(0x795767602000, 5640, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x795767602000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0-\0\1\0\0\0\0\0\0\0\0\0\0....", 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0....", 784, 64) = 784
pread64(3, "\4\0\0\0\0\0\0\0\5\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0....", 48, 848) = 48
pread64(3, "\4\0\0\0\0\24\0\0\0\0\3\0\0\0\0\0\0\0\0\325\31p\226\367\t\200\30\261\30\257\33\366c"..., 68, 896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0....", 784, 64) = 784
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x795767200000
mprotect(0x795767228000, 2023424, PROT_NONE) = 0
mmap(0x795767228000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x795767228000
mmap(0x7957673bd000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) = 0x7957673bd000
m2ntch0co@m2ntch0co-VirtualBox:~$ strace ls 2>&1 | grep openat
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache", O_RDONLY) = 3
m2ntch0co@m2ntch0co-VirtualBox:~$
```

b. Is 내에서 사용된 시스템 콜 조사 (시스템콜 포함)

execve

인자로 제공된 프로그램 실행

- **인자:** (실행 파일 경로명, 인수 배열, 환경 변수 배열)
 - 터미널에서 ls -l과 같이 명령어와 옵션을 입력하면 해당 값이 인수 배열로 입력됩니다.

- 해당 배열이 ls로 시작할 경우 ls 프로세스를 실행하는 방식으로 동작하는 것으로 추정됩니다.
 - **반환 값:** 성공 시 반환되지 않음 (현재 프로세스가 새 프로세스로 대체됨). 실패 시 -1 반환.
-

brk

데이터 세그먼트(힙)의 크기를 조절하여 메모리를 할당하거나 해제

단 인자를 NULL로 제공할 경우 현재 프로세스의 break 주소를 제공

> ls 프로그램 실행 후 초기 힙 위치 파악 용으로 사용된 것으로 추정됩니다.

- **인자:** (새로운 힙의 끝 주소)
 - **반환 값:** 성공 시 0. 실패 시 -1.
-

arch_prctl

아키텍처별로 프로세스, 스레드의 상태 설정

- **인자:** (설정 코드, 설정할 값의 주소)
 - 0x3001로 intel mpx 기능 설정을 요구했지만 현재 사용 중단된 기능으로 실패 값(-1)을 반환한 것 같습니다.
 - **반환 값:** 성공 시 0. 실패 시 -1.
-

access

파일의 존재 여부나 특정 접근 권한 확인

- **인자:** (경로명, 확인할 권한 모드)
 - ld.so.preload는 최우선적으로 메모리에 로드할 라이브러리를 담고 있음
 - 해당 파일이 없으므로 현재 로드할 라이브러리가 없음을 의미합니다.

- **반환 값:** 성공(권한 있음) 시 0. 실패(권한 없음 또는 오류) 시 -1.
-

openat

파일을 열고 파일 디스크립터를 반환합니다.

- **인자:** (기준 디렉터리 파일 디스크립터, 경로명, 열기 플래그)
 - /etc/ld.so.cache 파일은 필요한 라이브러리와 위치에 대한 정보 가짐
 - 해당 라이브러리를 열고 링킹하여 빠르게 ls 프로그램을 실행하게 함.
 - **반환 값:** 성공 시 새로운 파일 디스크립터(음수가 아닌 정수). 실패 시 -1.
-

read

파일 디스크립터에서 데이터를 읽습니다.

- **인자:** (파일 디스크립터, 데이터를 저장할 버퍼 주소, 읽을 바이트 수)
 - openat을 통해 fd를 얻었다면 해당 fd 기반으로 읽기 가능
 - 두번째 인자 주소를 시작점으로 하여 데이터를 읽고 사용자는 해당 주소를 통해 얻은 데이터를 통제하는 방식으로 동작함
 - 만약 읽을 바이트 수가 버퍼 크기보다 클 경우 segfault 가능성이 있음으로 sizeof 등의 조치가 필요해 보임.
 - **반환 값:** 성공 시 읽어 들인 바이트 수. 파일 끝이면 0. 실패 시 -1.
-

pread64

파일의 지정된 오프셋에서부터 데이터를 읽습니다. (파일의 현재 읽기 위치는 변경되지 않음)

- **인자:** (파일 디스크립터, 버퍼 주소, 읽을 바이트 수, 시작 오프셋)
 - 시작 오프셋을 직접 설정하여 읽는 방식으로 작업 후에도 기존 오프셋이

수정되지 않는 점에서 read와 차이가 있는 것으로 보입니다.

- 기본적으로 fd를 받아 버퍼 주소에 값을 읽는다는 점은 동일해보입니다.
 - **반환 값:** 성공 시 읽어 들인 바이트 수. 파일 끝이면 0. 실패 시 -1.
-

write

파일 디스크립터에 데이터를 씁니다.

- **인자:** (파일 디스크립터, 쓸 데이터가 있는 버퍼 주소, 쓸 바이트 수)
 - Read와 비슷하게 fd를 받아서 해당 파일에 내용을 추가하는 방식으로 동작합니다.
 - 파일에 데이터를 쓴 뒤 자동으로 파일 오프셋이 변경됨으로 로그 기록처럼 이전 내용을 기억해야 하는 경우 유용해 보입니다.
 - 덮어쓰기를 원할 경우 직접 오프셋을 수정하거나, pwrite64 사용가능
 - **반환 값:** 성공 시 쓰여진 바이트 수. 실패 시 -1.
-

close

열려 있는 파일 디스크립터를 닫습니다.

- **인자:** (파일 디스크립터)
 - Openat 혹은 open을 통해 파일 fd를 얻어서 활용한 뒤 해당 fd 번호와의 연결을 끊는 기능입니다.
 - 정상적으로 성공 시 해당 프로세스는 더 이상 해당 fd에 할당된 파일에 접근하지 않겠다는 의미를 가집니다.
 - **반환 값:** 성공 시 0. 실패 시 -1.
-

newfstat

파일 디스크립터를 통해 파일의 상태 정보(메타데이터)를 가져옵니다.

- **인자:** (파일 디스크립터, 파일 상태를 저장할 구조체 주소)
 - **반환 값:** 성공 시 0. 실패 시 -1.
-

statfs

파일 시스템의 상태 정보(총공간, 가용 공간 등)를 가져옵니다.

- **인자:** (경로명, 파일 시스템 상태를 저장할 구조체 주소)
 - **반환 값:** 성공 시 0. 실패 시 -1.
-

mmap

파일이나 장치를 메모리에 매핑합니다.

- **인자:** (매핑할 시작 주소, 길이, 보호 모드, 플래그, 파일 디스크립터, 오프셋)
 - **반환 값:** 성공 시 매핑된 메모리의 시작 주소. 실패 시 MAP_FAILED (일반적으로 (void *) -1).
-

munmap

메모리 매핑을 해제합니다.

- **인자:** (해제할 메모리 시작 주소, 길이)
 - **반환 값:** 성공 시 0. 실패 시 -1.
-

mprotect

메모리 영역의 접근 권한(읽기/쓰기/실행)을 변경합니다.

- **인자:** (메모리 시작 주소, 길이, 새로운 보호 모드)

- **반환 값:** 성공 시 0. 실패 시 -1.
-

getdents64

디렉터리에서 여러 개의 항목(파일, 하위 디렉터리)을 한 번에 읽습니다.

- **인자:** (디렉터리 파일 디스크립터, 항목 정보를 저장할 버퍼 주소, 버퍼 크기)
 - **반환 값:** 성공 시 읽어 들인 바이트 수. 디렉터리 끝이면 0. 실패 시 -1.
-

ioctl

장치(device)를 제어하기 위한 작업을 수행합니다. (예: 터미널 창 크기 얻기)

- **인자:** (파일 디스크립터, 요청 코드, 추가 인자)
 - **반환 값:** 요청에 따라 다르지만, 일반적으로 성공 시 0. 실패 시 -1.
-

prlimit64

프로세스의 자원 제한(열 수 있는 파일 개수 등)을 확인하거나 설정합니다.

- **인자:** (프로세스 ID, 리소스 종류, 새 제한 값 구조체 주소, 이전 제한 값 구조체 주소)
 - **반환 값:** 성공 시 0. 실패 시 -1.
-

getrandom

암호학적으로 안전한 난수를 생성하여 버퍼에 채웁니다.

- **인자:** (난수를 저장할 버퍼 주소, 요청할 바이트 수, 플래그)
- **반환 값:** 성공 시 버퍼에 채워진 바이트 수. 실패 시 -1.

set_tid_address

커널이 스레드 ID를 저장할 주소를 지정합니다. (스레드 종료 시 사용됨)

- **인자:** (스레드 ID를 저장할 포인터의 주소)
- **반환 값:** 호출한 스레드의 ID.

set_robust_list

스레드가 예기치 않게 종료될 때 자동으로 잠금 해제될 뮉텍스(futex) 목록을 설정합니다.

- **인자:** (목록의 헤드 포인터 주소, 구조체 크기)
- **반환 값:** 성공 시 0. 실패 시 -1.

rseq

원자적으로 실행될 필요가 있는 특정 코드 블록(Restartable Sequences)을 등록합니다.

- **인자:** (rseq 구조체 주소, 구조체 크기, 플래그, 시그니처)
- **반환 값:** 성공 시 0. 실패 시 -1.

futex

사용자 공간에서 빠른 잠금(lock)을 구현하기 위한 기본적인 대기 및 깨우기 기능을 제공합니다.

- **인자:** (futex 주소, 연산 종류, 값, 타임아웃 주소)
- **반환 값:** 연산 종류에 따라 매우 다양하며, 일반적으로 성공 시 0

c. ls 명령어에서 사용된 시스템콜 호출 횟수

```
m2ntch0co@m2ntch0co-VirtualBox:~$ strace -c ls
```

Makefile	gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu	linux			
buildroot	gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz	practice			
% time	seconds	usecs/call	calls	errors	syscall

36.46	0.000607	86	7		mprotect
12.07	0.000201	10	19		mmap
7.39	0.000123	15	8		openat
5.95	0.000099	19	5		read
4.62	0.000077	77	1		munmap
4.20	0.000070	23	3		brk
3.90	0.000065	32	2	2	statfs
3.42	0.000057	5	10		close
3.06	0.000051	5	9		newfstatat
2.94	0.000049	49	1		set_tid_address
2.70	0.000045	45	1		set_robust_list
2.28	0.000038	19	2	2	access
2.10	0.000035	35	1		prlimit64
2.10	0.000035	35	1		getrandom
2.04	0.000034	17	2	1	arch_prctl
1.74	0.000029	14	2		write
1.44	0.000024	12	2		getdents64
0.78	0.000013	13	1		rseq
0.54	0.000009	4	2		ioctl
0.24	0.000004	4	1		futex
0.00	0.000000	0	4		pread64
0.00	0.000000	0	1		execve

100.00	0.001665	19	85	5	total

시스템 콜	호출 횟수	시스템 콜	호출 횟수
mmap	19	write	2
close	10	getdents64	2
newfstat	9	munmap	1
openat	8	set_tid_address	1
mprotect	7	set_robust_list	1
read	5	prlimit64	1
pread64	4	getrandom	1
brk	3	rseq	1
statfs	2	ioctl	1
access	2	futex	1
arch_prctl	2	execve	1

d. ls 명령어에서 읽기 성공한 모든 파일 경로

```
m2ntch0co@m2ntch0co-VirtualBox:~$ strace ls 2>&1 | grep openat |grep -v "ENOENT"
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache", O_RDONLY) = 3
m2ntch0co@m2ntch0co-VirtualBox:~$
```

e. ls 명령어 동작 원리 분석

```
m2ntch0co@m2ntch0co-VirtualBox:~$ strace ls
execve("/usr/bin/ls", ["ls"], 0x7ffc9df61d40 /* 44 vars */) = 0
brk(NULL)                                = 0x5e3f9206d000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffc5fd9efc0) = -1 EINVAL (부적절한 인수)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x76788d9b2000
access("/etc/ld.so.preload", R_OK)        = -1 ENOENT (그런 파일이나 디렉터리가 없습니다)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=57876, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 57876, PROT_READ, MAP_PRIVATE, 3, 0) = 0x76788d9a3000
close(3)                                  = 0
```

- 터미널에서 ls가 입력된 뒤 프로세스 실행을 위한 기반 작업을 하는 부분입니다.
- Access와 openat으로 로드 및 링킹해야 하는 라이브러리를 확인하고 mmap으로 해당 내용을 메모리에 매핑한 뒤 기반 작업 수행을 종료하는 방식입니다.

- 현재 디렉토리를 기반으로 내부 항목을 읽어온 뒤 표준출력으로 해당 내용을 출력하는 ls의 기본 기능을 수행하는 영역입니다.

f. ls 동작에서 파일 목록 얻어오는 부분 명시 (설명 포함)

```
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
newfstatat(3, "", {st_mode=S_IFDIR|0750, st_size=4096, ...}, AT_EMPTY_PATH) = 0
getdents64(3, 0x5e3f92073a00 /* 24 entries */, 32768) = 856
```

- openat을 통해 현재 디렉토리(.)을 열고 3번 fd를 반환합니다.
- 해당 fd를 기반으로 getdents64 시스템 콜로 파일, 하위 디렉토리 항목을 읽어옵니다.

g. ls 동작에서 파일 목록 출력하는 부분 명시 (설명 포함)

```
ioctl(1, TCGETS, {B38400 opost isig icanon echo ...}) = 0
ioctl(1, TIOCGWINSZ, {ws_row=19, ws_col=104, ws_xpixel=0, ws_ypixel=0}) = 0
```

```
write(1, "buildroot\t\t\t\t\t linux \353\213\244\354\232\264\353"... , 60buildroot
      linux 다운로드 비디오 템플릿
) = 60
write(1, "gcc-arm-10.3-2021.07-x86_64-aarc"... , 79gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-
gnu snap 문서 사진
) = 79
write(1, "gcc-arm-10.3-2021.07-x86_64-aarc"... , 89gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-
gnu.tar.xz 공개 바탕화면 음악
) = 89
```

- ioctl 시스템 콜을 통해 현재 터미널 창 크기를 확인합니다. 이를 통해 받은 파일 목록을 어떻게 배열할 지 결정합니다.
- write 시스템 콜을 통해 받은 파일 목록을 표준 출력(1)에 작성합니다.